
CSE Web Data Collection

Document for 25ada072 - Pasindu

Your part: Getting the data from the site, and Selenium

Everything behind your part of the project - the ideas, the reasons, and the code - explained step by step, from the beginning.

DA 2009 - Data Collection Methods II

Assignment 2 (Group Project)

University of Colombo

25ada072 - Pasindu | 25ada073 - Sasini | 25ada141 - Salaama | 24ada076 - Nuhan

Contents

How to use this document	3
The project in simple words	4
How the web works	6
The ideas behind your part	9
Reading Python	13
Your file: <code>src/api_client.py</code>	19
Your file: <code>src/selenium_scraper.py</code>	24
Your files: <code>config.py</code> and <code>warm_cache.py</code>	28
How your part fits into the whole project	30
Ethics - what every member must know	31
Showing your part in the viva	32
Practice questions	34
Word list	41

How to use this document

This document explains your part of the project from the very beginning. It does not assume you have seen any of it before. Each chapter builds on the one before it, so read it in order the first time.

Your files are:

- `src/api_client.py`
- `src/selenium_scraper.py`
- `config.py`
- `warm_cache.py`

You will see four kinds of coloured box:

Think of it like this. a comparison with something from everyday life, to make a new idea easier to picture.

Remember: the one thing to take away from a section. If you only read the green boxes, you will still know the main points.

Be careful: a trap, or something an examiner might test you on.

Try it yourself. something to type on the computer, and what should happen.

Grey boxes in a typewriter font are **code** - real lines copied from our project files, not made-up examples.

A plan for the week before the viva

Day	What to do
1	Read chapters 1 to 3. They explain the project and the ideas behind your part.
2	Read the chapter on reading Python, with your own files open beside it.
3 and 4	Read the chapters on your files, one at a time. Run each file as the 'Try it yourself' boxes show.
5	Read how your part fits into the whole project, and the ethics chapter.
6	Practise the viva chapter out loud - say your 30-second explanation until it feels natural.
7	Go through the practice questions at the end. Cover the answer and try to say it first.

The project in simple words

What a stock exchange is

A company can split its ownership into many small pieces called **shares**. If you buy one share of a bank, you own a tiny piece of that bank.

A **stock exchange** is the marketplace where people buy and sell those shares. The price of a share goes up when many people want to buy it, and down when many people want to sell it. The **Colombo Stock Exchange** - CSE for short - is Sri Lanka's stock exchange, and its website is www.cse.lk.

Think of it like this. a stock exchange is like a big weekly fair where, instead of vegetables, people trade small pieces of companies. The website is the notice board at the entrance that shows today's prices.

What we were asked to do

Our group was given the CSE website and asked to collect data from it with a computer program, using the methods taught in the course, and to do it **ethically** - politely, legally, and without causing the website any trouble.

What we collect

What	What it means	Example
Share prices	What every company traded at today	Sampath Bank: Rs 141.00
Company details	Facts about one company	Its total value, its highest and lowest price this year
Announcements	Notices companies publish, often as PDF files	'A dividend will be paid', 'A meeting will be held'

After collecting, we tidy the data, check it makes sense, draw charts, and save it as files that other people can open.

Who did what

Member	Part of the project
24ada076 - Nuhan	Ethical scraping and robots.txt, BeautifulSoup, and reading PDFs
25ada072 - Pasindu (this is you)	Getting the data from the site, and Selenium
25ada073 - Sasini	Crawling for documents, and cleaning the data
25ada141 - Salaama	OCR, saving the files, and the charts

The app itself (app.py) was put together by all four of us. Each tab in the app is labelled with the member whose part it shows.

The one surprise that shaped the whole project

The first thing we tried was the ordinary method from the course: download the web page and pick the prices out of it. It found almost nothing - about 24 letters of text, and no tables at all.

The reason is that the CSE website is built with **JavaScript**. The page arrives nearly empty, and a small program inside your browser then fetches the prices and fills them in. A plain download never runs that program, so it never sees the prices.

So we opened the site in a browser, pressed **F12**, and watched the **Network** tab, which lists everything the page asks for. It showed the page fetching its prices from addresses such as `https://www.cse.lk/api/tradeSummary`. We read those addresses directly instead, and one request gives us every company at once.

Remember: the CSE page is built by JavaScript, so we read the data from the same addresses the page itself uses. That one decision explains why the project is built the way it is.

How the web works

Before looking at any code, it helps to understand what actually happens when a computer 'visits' a website. There are only a few ideas, and every part of our project uses them.

Requests and responses

When you open a web page, your computer sends a message to another computer, called a **server**, asking for that page. This message is a **request**. The server sends something back - the page, a file, or some data. That is the **response**.

Think of it like this. writing a letter and getting a reply. You write to an address and ask for something. A little later, a reply arrives in your letterbox. Every single thing a website shows you - each picture, each price - arrives this way, one letter and one reply at a time.

Addresses - URLs

Every page and file on the web has an address, called a **URL**. For example:

```
https://www.cse.lk/api/tradeSummary
```

Part	What it means
https://	How to talk to the server. The 's' means the conversation is locked so nobody can read it on the way.
www.cse.lk	Which server - the CSE website.
/api/tradeSummary	Which thing on that server we want. This part is called the path .

GET and POST - two ways of asking

There are two common kinds of request. A **GET** request simply asks for something: 'please give me this page'. A **POST** request sends some information along with the question, like filling in a form before handing it over.

Most of the CSE data addresses only answer **POST** requests - if you send a GET, they reply with an error. That is their rule, not ours, and we found it by trying both.

Status codes - the result stamped on every reply

Every response comes with a three-digit number that says how it went. These were taught in Week 2:

Code	Meaning	In everyday words
200	OK	Here is what you asked for.
301	Moved Permanently	That has moved - look over there.
403	Forbidden	You are not allowed to have this.
404	Not Found	There is no such page.

Code	Meaning	In everyday words
500	Internal Server Error	Something broke on our side.

Headers - the envelope

Along with every request travel a few lines of extra information called **headers**. One of them, the **User-Agent**, says what program is asking. A browser sends something like 'Chrome on Windows'. Our project sends its own name, the university, the course, and an email address.

Think of it like this. the User-Agent is like writing your name and address on the back of the envelope. The receiver knows who wrote, and can write back if there is a problem.

HTML - what a web page is made of

A web page is a text file written in **HTML**. It uses **tags** - words inside angle brackets - to label each part of the page:

```
<html>
  <head>
    <title>Trade Summary</title>
  </head>
  <body>
    <h1>Today's prices</h1>
    <table>
      <tr> <th>Company</th> <th>Price</th> </tr>
      <tr> <td>Sampath Bank</td> <td>141.00</td> </tr>
    </table>
  </body>
</html>
```

Tags usually come in pairs: <title> opens and </title> closes. Whatever sits between them is the content. Tags sit inside other tags, like boxes inside boxes: the table is inside the body, each row <tr> is inside the table, and each cell <td> is inside a row.

JSON - data in a tidy shape

Websites do not only send pages. They also send plain **data**, and the most common format for that is **JSON**. JSON is made of **names and values**, grouped with curly brackets { }, and lists grouped with square brackets []. This is a real piece of what CSE sends us:

```
{
  "symbol": "SAMP.N00000",
  "name": "SAMPATH BANK PLC",
  "price": 141.0,
  "change": 1.0
}
```

Think of it like this. JSON is like a neatly filled-in form. Every box has a label ('name', 'price') and an answer. A program can read it straight away, with no need to search through the decorations of a web page.

Static and dynamic pages

	Static page	Dynamic page
Where the content is	Already written inside the HTML file	Added by JavaScript after the page arrives
Can a plain download see it?	Yes	No
Example	A list of university departments	CSE's live prices

Think of it like this. a static page is a printed menu - everything is already on it. A dynamic page is an empty chalkboard; a waiter writes today's specials on it after you sit down. If you take a photo the moment you walk in, you photograph an empty board.

Try it yourself. open <https://www.cse.lk/equity/trade-summary> in Chrome or Edge. Right-click and choose **View Page Source** - this shows what the server sent, and you will not find the prices in it. Now right-click a price and choose **Inspect** - this shows the page after JavaScript ran, and the prices are there.

The ideas behind your part

Your part is where the project actually gets its numbers. It rests on two ideas: **APIs** - addresses that hand out data instead of pages - and **browsers**, which run JavaScript to build a page. This chapter explains both before we look at any code.

What an API is

API stands for Application Programming Interface. That sounds complicated, but the idea is simple: it is a way for one program to ask another program for data. You send a request, and you get a reply - usually in JSON.

Think of it like this. the Week 5 lecture compared an API to a waiter. You tell the waiter what you want. The waiter takes the order to the kitchen. The kitchen cooks it, and the waiter brings it back to your table. You never go into the kitchen yourself. An API is the waiter between your program and the website's data.

How a website talks to its data

Behind every website there is a **database** - a store of all its information. The lecture described how a page gets data out of it:

```
The page (in your browser)
  | 1. asks for something
  v
The API (/api/tradeSummary)
  | 2. asks the database for you
  v
The database
  | 3. finds the data
  v
The API
  | 4. sends it back as JSON
  v
The page shows it
```

Why websites have APIs

The Week 5 lecture gave these reasons:

- **So the site's own pages can load data** without reloading the whole page. This is CSE's reason - their trade summary page calls `/api/tradeSummary` itself.
- To share data with developers safely.
- To let others build apps with their data.
- To reduce the load on the website, because data can be fetched without opening full pages.

JSON, the shape of the replies

Every reply from a CSE data address is JSON. You met the basics in the chapter on how the web works. Here is a real record for one company, exactly as CSE sent it:

```
{
  "id": 266,
  "name": "SAMPATH BANK PLC",
```

```

    "symbol": "SAMP.N00000",
    "price": 141.0,
    "previousClose": 140.25,
    "change": 0.75,
    "percentageChange": 0.5347593582887701,
    "high": 141.25,
    "low": 140.0,
    "turnover": 13244976.75,
    "lastTradedTime": 1790326797431,
    "marketCap": 165350807160.0
    ... (23 fields in all)
}

```

Field	Meaning
price	What one share sold for today, in rupees
previousClose	The price at the end of the last trading day
change	The difference: 141.00 - 140.25 = 0.75
percentageChange	That change as a percentage of the old price
turnover	Total money that changed hands for this company today
lastTradedTime	When the last trade happened - as a number (explained below)
marketCap	The whole company's value: share price times the number of shares

The whole reply is one big wrapper holding a list of these records - one for every company that traded:

```

{
  "reqTradeSummery": [
    { ...ABANS ELECTRICALS PLC... },
    { ...ABANS FINANCE PLC... },
    ...
    { ...SAMPATH BANK PLC... },
    ... (284 records on the day we checked)
  ]
}

```

Notice reqTradeSummery is misspelled - 'Summery' instead of 'Summary'. That is CSE's spelling, so our code has to use it exactly. The same goes for topLooses and companyInfoSummery. When you read somebody else's API, you use their names, spelling mistakes and all.

Dates as big numbers

lastTradedTime is 1790326797431. That is not a mistake. It is the number of **milliseconds since 1 January 1970** - a common way for computers to store a moment in time, called **epoch time**. Divide by 1000 to get seconds, and Python can turn it into a normal date. Your function to_datetime() does exactly that.

How we found CSE's data addresses

CSE does not publish a list of its data addresses. We found them the way the Week 1 lecture taught: by watching the page work.

1. Open <https://www.cse.lk/equity/trade-summary> in Chrome or Edge.

2. Press **F12**. The developer tools open at the side or bottom.
3. Click the **Network** tab. It lists every request the page makes.
4. Click the **Fetch/XHR** filter, so only data requests are shown - not pictures and fonts.
5. Press **F5** to reload the page, and watch the list fill up.
6. Click the request called `tradeSummary`. The **Headers** section shows its address and that it was sent as **POST**. The **Response** or **Preview** section shows the JSON it got back.

Think of it like this. the Network tab is like standing next to the waiter's hatch in a restaurant, writing down every order that goes into the kitchen. You learn exactly what to ask for - and then you can ask for it yourself.

Try it yourself. do these six steps yourself before the viva. Being able to say 'I pressed F12 and watched the Network tab' - and to show it - is the most convincing thing you can say about your part.

Why an API is better than scraping the page

The Week 6 lecture said it plainly: 'Use APIs where available - an official API is generally more reliable and structured than scraping.' For CSE the difference is large:

	Reading the data address	Scraping the page in a browser
Companies from one go	All of them - 284	About 25 - the table is split into pages
Time	About 1 second	About 5 seconds
Size of reply	One JSON file	142 KB of HTML, plus scripts, fonts and images
Work for CSE's server	Send one piece of data	Send the whole page - and then the browser asks for the same data anyway

That last row is the ethical point. Loading the page does not avoid the data address - the page calls it too. Reading it directly just skips all the extra work.

The limits of an API

The Week 5 lecture listed the weaknesses of APIs too. Here is how each one applies to us:

Limitation	Does it affect us?
Limits on how many requests you may send	CSE sets none that we know of - we wait 1.5 seconds anyway
May need a key or an account	No - CSE's addresses are open
May give less data than the website shows	No - it is the same data the page shows
Can change without warning	Yes - these addresses are not officially published, so CSE could change them any day

Remember: we found CSE's data addresses by watching the Network tab, and we use them because they are faster, give every company at once, and are less work for CSE's server than loading the page.

Browsers, JavaScript and Selenium

When a browser opens the CSE page, it does much more than download a file:

1. It downloads the HTML - the nearly empty shell.
2. It downloads the JavaScript files the HTML points to.
3. It **runs** the JavaScript.
4. The JavaScript calls `/api/tradeSummary` - the same address your code uses.
5. The JavaScript builds the table and puts it on the page.

The page, as it is after all that, is called the **DOM**. requests only ever does step 1, which is why it sees nothing. A real browser does all five.

Selenium is a tool that lets a program control a real web browser - open pages, click, type, scroll - exactly as a person would. The Week 6 lecture showed how it works:

```
Your Python script
    | 'open this page'
    v
The driver (msedgedriver / chromedriver)
    | passes the order on
    v
The real browser (Edge / Chrome)
    | opens the page, runs the JavaScript
    v
The finished page comes back to your script
```

The **driver** is a small program that sits between Python and the browser, turning Python's orders into actions. Each browser needs its own driver, and it must match the browser's version. The lecture described downloading it by hand - but since Selenium 4.6, a part called **Selenium Manager** finds and downloads the right driver by itself.

Think of it like this. requests is like asking for a photocopy of a newspaper before it has been printed. Selenium sends a robot to sit at a real computer, open the paper, wait for the ink to dry, and then take a photo.

Selenium word	Meaning
<code>driver.get(url)</code>	Open this page in the browser
Headless	Run the browser invisibly, with no window - faster, and it does not pop up during a demonstration
<code>WebDriverWait</code>	Wait until something appears on the page - the lecture's 'explicit wait'
<code>driver.page_source</code>	The page's HTML as it is now, after the JavaScript has run
<code>driver.save_screenshot()</code>	Save a picture of what the browser shows
<code>driver.quit()</code>	Close the browser completely

Reading Python

To explain your code, you first need to be able to read it. This chapter explains every piece of Python that appears in your files. Every example is a real line from your own code, so by the end of the chapter you will recognise all of it.

Try it yourself. open `src/api_client.py` in VS Code or Notepad and keep it beside this document while you read. Find each example in the file as you go.

Comments - notes for people

A line that starts with `#` is a **comment**. Python ignores it completely. It is there for people reading the code. Our files are full of comments explaining why each part exists.

From your code:

```
# The reply has several parts. The company details are under
# reqSymbolInfo.
```

From `company_info()`. It explains the line below it to a reader. Python skips both lines.

Variables - labelled boxes

A **variable** is a name that holds a value. The `=` sign means 'put the thing on the right into the box on the left'. It does not mean 'equals' the way it does in maths.

Think of it like this. a variable is a labelled box. `price = 141.0` means: take a box, write 'price' on the label, and put 141.0 inside.

From your code:

```
url = f"{config.API_BASE}/{endpoint}"
```

From `call()`. The finished address is put in a box called `url`, ready to be sent.

Text, and filling in blanks with f-strings

Text in Python is written inside quote marks, like `"hello"`. It is called a **string**. A string that starts with `f` is an **f-string**: anything inside curly brackets `{ }` is replaced with the value of that variable.

Think of it like this. an f-string is a form letter with blanks. `f"Dear {name}"` fills in the blank with whatever name holds.

From your code:

```
url = f"{config.API_BASE}/{endpoint}"
```

The same line, as an f-string. `config.API_BASE` holds `https://www.cse.lk/api`. If `endpoint` holds `tradeSummary`, the result is `https://www.cse.lk/api/tradeSummary`.

Numbers, and True and False

Python has whole numbers like 30, numbers with a decimal point like 1.5, and two special values: True and False. None means 'nothing here yet'.

From your code:

```
DELAY = 1.5
TIMEOUT = 30
```

From `config.py`. The polite pause between requests, and how many seconds to wait for a reply before giving up. Names in capital letters are a Python habit meaning 'a setting - do not change this while the program runs'.

Lists - things in order

A **list** holds several values in order, inside square brackets. `.append()` adds one more to the end. `[]` on its own is an empty list, ready to be filled.

Think of it like this. a list is a numbered shopping list. You can add items to the end, and read them back in order.

From your code:

```
rows = []
for number, symbol in enumerate(symbols, start=1):
    ...
    info = self.company_info(symbol)
    if info:
        rows.append(info)
```

From `several_companies()`. Start with an empty list, then add each company's details to it, one at a time.

Dictionaries - values with labels

A **dictionary** holds values under names, inside curly brackets: `{"name": value}`. You get a value back by its name: `reply["ok"]`. The method `.get(name, default)` does the same, but gives back the default instead of crashing if the name is missing.

Think of it like this. a dictionary is a set of labelled drawers. You open a drawer by reading its label, not by counting from the top.

From your code:

```
ENDPOINTS = {
    "tradeSummary": {"method": "POST", "desc": "Trade data for every company"},
    "allSecurityCode": {"method": "GET", "desc": "List of all listed companies"},
    ...
}
```

From `config.py`. A dictionary of dictionaries: under each address name is a smaller dictionary saying whether it needs GET or POST. Your `call()` reads it with `config.ENDPOINTS.get(endpoint,`

`{}) .get("method", "POST")` - 'find this address; if it is not listed, use an empty dictionary; then find its method, or use POST if none is given'.

Functions - recipes

A **function** is a named set of steps. `def` starts one. The names in brackets are its **parameters** - the ingredients it needs. `return` hands back the result. The function does nothing until somebody **calls** it by writing its name followed by brackets.

Think of it like this. a function is a recipe. The parameters are the ingredients, the indented lines are the steps, and `return` is the finished dish.

From your code:

```
def company_info(self, symbol):
    reply = self.call("companyInfoSummary",
                      data={"symbol": symbol})
    if not reply["ok"]:
        return {}
    return dict(reply["data"].get("reqSymbolInfo", {}))
```

The ingredient is a company symbol such as `SAMP.N0000`. The function sends it with the request and returns that company's details - or an empty dictionary `{}` if something went wrong.

if and else - making decisions

`if` runs the indented lines only when a condition is true. `else` runs when it is not. `not` turns true into false and the other way round.

From your code:

```
if not reply["ok"]:
    return pd.DataFrame()
```

From `sectors()` and several others. If the request failed, hand back an empty table straight away, so the app shows nothing instead of crashing.

for loops - doing the same thing for each item

A **for loop** repeats its indented lines once for every item in a list or table. `enumerate(..., start=1)` also counts them for you: 1, 2, 3.

Think of it like this. a for loop is a teacher going down the register, doing the same thing for each name.

From your code:

```
for number, symbol in enumerate(symbols, start=1):
```

From `several_companies()`. Go through the list of symbols one by one, counting 1, 2, 3 so the progress bar can say 'Getting SAMP.N0000 (3 of 15)'.

import - taking a toolbox off the shelf

Python comes with some tools built in. For everything else we **import** a library - a toolbox somebody else wrote. `import pandas as pd` means 'load pandas, and let me call it pd for short'. `from X import Y` takes just one tool out of the box.

From your code:

```
from datetime import datetime
import pandas as pd
from selenium import webdriver
from bs4 import BeautifulSoup
```

Your files use `datetime` to read epoch times, `pandas` to make tables, `Selenium` to drive a browser, and `BeautifulSoup` to read the browser's finished page.

try and except - a safety net

Code inside `try` is attempted. If anything goes wrong, Python jumps to `except` instead of crashing. We use this wherever the internet or a file might fail.

Think of it like this. `try` and `except` is a trapeze artist with a safety net. They attempt the jump; if they fall, the net catches them and the show goes on.

From your code:

```
try:
    return datetime.fromtimestamp(float(value) / 1000)
except (ValueError, OSError, TypeError):
    return None
```

From `to_datetime()`. If the value cannot be turned into a date - it is text, or far too big - give back `None` instead of stopping the whole table from loading. Naming the exact errors in brackets means only those are caught.

Classes, objects and self

A **class** is a design for making things that hold both information and actions. The thing you make from the design is an **object**. Functions written inside a class are called **methods**. `__init__` is the special method that runs once, when the object is first made - it sets it up. Inside a class, **self** means 'this particular object'.

Think of it like this. a class is a cake tin, and each object is a cake made from it. `self` means 'this cake' - so `self.delay` is this cake's own delay, not somebody else's.

From your code:

```
class CSEApi:

    def __init__(self, scraper=None):
        self.scraper = scraper or Scraper()
```

From `api_client.py`. When the app makes a `CSEApi()`, it is given a `Scraper` - Nuhan's class that sends requests politely. `scraper` or `Scraper()` means 'use the one I was given, or make a new one'. Every method then sends its requests through `self.scraper`.

pandas DataFrames - tables in Python

A **DataFrame** is a table with rows and named columns, like a sheet in Excel. `pandas` is the library that provides it. `pd.DataFrame(rows)` builds one from a list of dictionaries, `df.head(5)` shows the first five rows, and `df["price"]` picks out one column.

From your code:

```
df = pd.DataFrame(records)
...
df[column] = df[column].apply(to_datetime)
```

From `make_dataframe()`. The first line turns the list of company records into a table - one row per company, one column per field. The second runs `to_datetime()` on every value in a date column. `.apply()` means 'do this to each value'.

isinstance - what kind of thing is this?

`isinstance(x, list)` answers `True` if `x` is a list. Your `get_records()` uses it because different CSE addresses send different shapes:

```
if isinstance(reply, list):
    ...
if isinstance(reply, dict):
    ...
```

try and finally - always tidy up

`finally` runs at the end whatever happened - success or error. Your Selenium code uses it to make sure the browser always closes:

```
try:
    driver.get(url)
    ...
finally:
    driver.quit()
```

Without it, one error would leave an invisible browser running in the background, using memory, and after a few failed runs there would be several.

Functions without brackets

Writing a function's name **with** brackets runs it: `api.aspi()`. Writing it **without** brackets just refers to it, so it can be stored and run later. Your `warm_cache.py` makes a list of jobs this way:

```
jobs = [
    ("Market status", api.market_status),
    ("Index", api.aspi),
    ...
]
for name, job in jobs:
    result = job()
```

Nothing runs while the list is being made. Each job runs later, at `job()`.

Running a file

Every file in our project can be run on its own to show what it does. You open a terminal in the project folder, switch on the project's Python, and ask Python to run the file:

```
cd D:\DA2009\CSE-Scraping-Project
.venv\Scripts\activate
python -m src.ethics
```

`.venv` is the project's own copy of Python with all the libraries installed. `-m src.ethics` means 'run the file `ethics.py` inside the `src` folder'. The part at the bottom of each file that starts `if __name__ == "__main__":` is what runs when you do this.

Your file: `src/api_client.py`

This is where the project gets its numbers. It asks CSE's data addresses for information and turns each JSON reply into a table.

The file has three small helper functions at the top, and then one class, `CSEApi`, with one method for each kind of data we want.

Part 1 - The three helpers

`to_datetime()`: big numbers into dates

```
def to_datetime(value):
    if value is None:
        return None
    try:
        return datetime.fromtimestamp(float(value) / 1000)
    except (ValueError, OSError, TypeError):
        return None
```

1790326797431 divided by 1000 is the number of seconds since 1970. `datetime.fromtimestamp()` turns seconds into a normal date and time. `float(...)` makes sure the value is a number first.

`fromtimestamp()` gives the time in the computer's own time zone. On our laptops in Sri Lanka that is Sri Lanka time, which is what we want. Sasini's cleaner converts the share price times to Colombo time on purpose, so those are right on any computer.

`get_records()`: digging out the list

The CSE addresses do not all reply in the same shape. Some send a plain list. Some wrap the list in a dictionary under a name like `reqTradeSummery`. A few send a list inside another list. This function copes with all of them, so the rest of the code does not have to:

```
def get_records(reply, key=None):
    if reply is None:
        return []

    if isinstance(reply, list):
        # A few replies are a list inside another list.
        if len(reply) == 1 and isinstance(reply[0], list):
            return reply[0]
        return reply

    if isinstance(reply, dict):
        if key and key in reply:
            return reply[key]
        for value in reply.values():
            if isinstance(value, list):
                return value
        return [reply]

    return []
```

If the reply looks like	It gives back
[{...}, {...}] - a plain list	The list as it is

If the reply looks like	It gives back
[[{...}, {...}]] - a list inside a list	The inner list
{"reqTradeSummery": [...]} - a wrapper	The list under the name we asked for
{"status": "Market Closed"} - one record	That record, inside a list of one
None - nothing	An empty list

Think of it like this. parcels arrive in different packaging - a bag, a box, a box inside a box. `get_records()` unwraps each one, so what lands on the table is always the same: a list of records.

make_dataframe(): the list becomes a table

```
def make_dataframe(records, date_columns=()):
    if not records:
        return pd.DataFrame()

    df = pd.DataFrame(records)

    for column in date_columns:
        if column in df.columns:
            df[column] = df[column].apply(to_datetime)

    return df
```

`pd.DataFrame(records)` does most of the work in one line: a list of dictionaries becomes a table, one row per dictionary, one column per name. Then any columns we name as dates are converted with `to_datetime()`.

284 dictionaries like the Sampath Bank one become a table like this:

	symbol	name	price	change	percentageChange
0	ABAN.N0000	ABANS ELECTRICALS PLC	1058.25	16.0	1.535140
1	AFSL.N0000	ABANS FINANCE PLC	84.50	1.0	1.197605
2	AEL.N0000	ACCESS ENGINEERING PLC	80.10	0.0	0.000000
3	ACL.N0000	ACL CABLES PLC	92.50	-0.8	-0.857449
4	APLA.N0000	ACL PLASTICS PLC	159.50	0.5	0.314465

Part 2 - call(), sending one request

```
class CSEApi:

    def __init__(self, scraper=None):
        self.scraper = scraper or Scraper()

    def call(self, endpoint, data=None):
        method = config.ENDPOINTS.get(endpoint, {}).get("method",
                                                         "POST")

        url = f"{config.API_BASE}/{endpoint}"
        return self.scraper.fetch(url, method=method, data=data,
                                  save_as=endpoint)
```

Every method in the class sends its request through `call()`. It does three things:

1. Looks up whether this address needs GET or POST, in the ENDPOINTS table in config.py.
2. Builds the full address, such as `https://www.cse.lk/api/tradeSummary`.
3. Hands it to `self.scrapper.fetch()` - Nuhan's function, which checks robots.txt, waits 1.5 seconds, sends the request with our User-Agent, and records it. `save_as=endpoint` means the reply is also saved as `data/cache/tradeSummary.json`.

Remember: your code never talks to the internet directly. It asks Nuhan's `fetch()` to do it, so every one of your requests follows the ethical rules automatically.

Part 3 - One method for each kind of data

Most methods follow the same three-line pattern. Here is the main one:

```
def trade_summary(self):
    reply = self.call("tradeSummary")
    if not reply["ok"]:
        self.scraper.log.add(f"Could not get trade summary: "
                            f"{reply['error']}")
        return pd.DataFrame()

    df = make_dataframe(get_records(reply["data"],
                                   "reqTradeSummary"))
    self.scraper.log.add(f"Got trade data for {len(df)} companies")
    return df
```

1. **Ask** - `self.call("tradeSummary")`.
2. **Check** - if it failed, write a message in the log and return an empty table.
3. **Unwrap and tabulate** - `get_records()` digs out the list, `make_dataframe()` makes the table.

The whole set:

Method	Address	What it gives back
<code>market_status()</code>	<code>marketStatus</code>	'Market Open' or 'Market Closed'
<code>aspi()</code>	<code>aspiData</code>	The All Share Price Index - one number for the whole market
<code>market_summary()</code>	<code>marketSummary</code>	Today's total turnover and volume
<code>sectors()</code>	<code>allSectors</code>	A table of sector indices - Banks, Energy and so on
<code>trade_summary()</code>	<code>tradeSummary</code>	The main table - every company that traded
<code>top_gainers()</code>	<code>topGainers</code>	The biggest risers
<code>top_losers()</code>	<code>topLooses</code>	The biggest fallers (CSE's spelling)
<code>all_companies()</code>	<code>allSecurityCode</code>	Every listed company and its symbol
<code>company_info(symbol)</code>	<code>companyInfoSummary</code>	Details for one company
<code>announcements()</code>	<code>approvedAnnouncement</code>	Company announcements
<code>circulars()</code>	<code>circularAnnouncement</code>	CSE circulars, with PDF links

company_info() - sending a value with the request

```
def company_info(self, symbol):
    reply = self.call("companyInfoSummery",
                      data={"symbol": symbol})
    if not reply["ok"]:
        return {}
    return dict(reply["data"].get("reqSymbolInfo", {}))
```

This address has to know which company you mean, so the symbol is sent **with** the POST request, as form data: data={"symbol": "SAMP.N0000"} - like filling in one box on a form before handing it in.

The reply has four parts: reqSymbolBetaInfo, reqTagsLogo, reqLogo and reqSymbolInfo. We found which one held the prices by printing the reply and looking. The details are in reqSymbolInfo, so that is the part we keep.

several_companies() - slow on purpose

```
def several_companies(self, symbols, progress=None):
    rows = []
    for number, symbol in enumerate(symbols, start=1):
        if progress:
            progress(number / len(symbols),
                     f"Getting {symbol} ({number} of {len(symbols)})")
        info = self.company_info(symbol)
        if info:
            rows.append(info)
    return make_dataframe(rows)
```

Each company is a separate request, and each one waits its 1.5 seconds - so 15 companies take a little over 20 seconds. That is deliberate. This is the only loop in the project that sends many requests in a row, so it is where politeness matters most. progress(. . .) moves the progress bar in the app.

circulars() - building full PDF addresses

```
df = make_dataframe(get_records(reply["data"],
                                "reqCircularAnnouncement"))

if not df.empty and "path" in df.columns:
    df["pdf_url"] = config.CDN_BASE + df["path"].astype(str)
```

Each circular comes with only part of its PDF's address, such as upload_report_file/abc.pdf. The PDFs live on a different server, so we add https://cdn.cse.lk/cmt/ in front. Adding two text columns with + in pandas joins them row by row, making a new column pdf_url. Sasini's crawler then uses those links.

What you see when you run it

Try it yourself. run your file on its own:

```
python -m src.api_client
```

```
Market status: Market Closed
ASPI index    : 21,037.35 (-29.54)
```

```
Trade summary - first 5 companies
      symbol          name    price  change
0  ABAN.N00000  ABANS ELECTRICALS PLC  1058.25   16.0
...

One company
SAMPATH BANK PLC - 141.0

17:48:03  POST https://www.cse.lk/api/marketStatus - status 200
17:48:04  POST https://www.cse.lk/api/aspiData - status 200
17:48:05  POST https://www.cse.lk/api/tradeSummary - status 200
17:48:05  Got trade data for 284 companies
```

Look at the times at the bottom: each request is at least a second and a half after the one before. That is Nuhan's `wait()` working on your requests.

Your file: src/selenium_scraper.py

Opens the CSE page in a real browser, so the JavaScript runs and the prices actually appear - then reads the finished page.

This file is not how the project collects its data - the API is. It is here to show the browser method works, to prove the API gives the same numbers a visitor sees, and as a backup if the data addresses ever change.

Part 1 - start_browser()

```
def start_browser(headless=True):
    from selenium import webdriver

    browsers = [
        ("Edge", webdriver.Edge, webdriver.EdgeOptions),
        ("Chrome", webdriver.Chrome, webdriver.ChromeOptions),
        ("Firefox", webdriver.Firefox, webdriver.FirefoxOptions),
    ]

    for name, browser, options_class in browsers:
        try:
            options = options_class()
            if headless:
                options.add_argument("--headless=new")
                options.add_argument("--window-size=1920,1080")
                options.add_argument(f"--user-agent={config.USER_AGENT}")

            return browser(options=options), name
        except Exception:
            continue

    return None, "No browser could be started. ..."
```

- The lecture used Chrome. The laptop we built this on has only Microsoft Edge, so the code tries Edge first, then Chrome, then Firefox. It works on anybody's computer.
- The list holds three **tuples** - groups of three things each: a name, the browser, and its settings type.
- --headless=new - run without a window.
- --window-size=1920,1080 - pretend the screen is a normal laptop size, so the page lays itself out properly.
- --user-agent=... - the same honest name as the rest of the project, so CSE sees one consistent visitor.
- If a browser will not start, continue moves on and tries the next one.

Part 2 - load_page()

```
driver, browser_name = start_browser(headless)
...
try:
    driver.get(url)

    try:
        WebDriverWait(driver, wait_seconds).until(
            expected_conditions.presence_of_element_located(
                (By.TAG_NAME, "table"))
        )
```

```

except Exception:
    pass # No table appeared, but we can still look at the page.

time.sleep(2) # Let the last few rows settle.

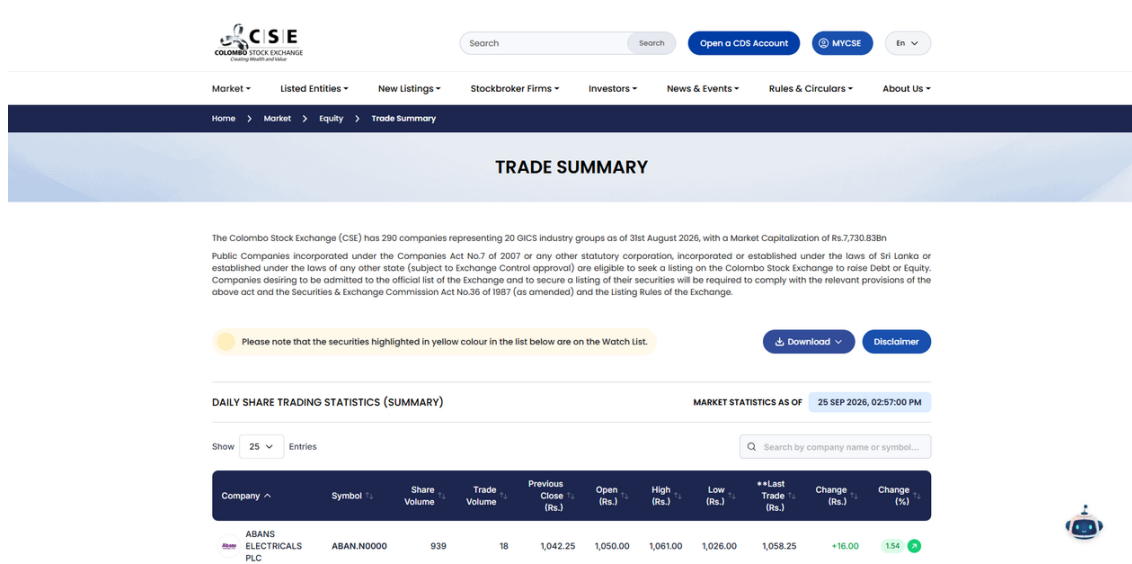
html = driver.page_source
driver.save_screenshot(screenshot)

soup = BeautifulSoup(html, "html.parser")
words = soup.get_text(strip=True)
tables = soup.find_all("table")
...
finally:
    try:
        driver.quit()
    except Exception:
        pass

```

Step by step:

1. `driver.get(url)` - the browser opens the page. At this moment the table does not exist yet - JavaScript is still fetching the data.
2. `WebDriverWait(driver, 15).until(...)` - wait, up to 15 seconds, until a `<table>` tag appears on the page. This is the lecture's **explicit wait**. By `TAG_NAME` means 'find it by its tag name'.
3. `time.sleep(2)` - a short extra pause so the last rows finish appearing.
4. `driver.page_source` - the page's HTML **as it is now**, after JavaScript. This is the key line. It is the difference from `response.text` in the requests version, which is the page before JavaScript.
5. `save_screenshot()` - a picture of what the browser saw.
6. From there it is ordinary BeautifulSoup, the same methods as in Nuhan's file.
7. `finally: driver.quit()` - always close the browser, even if something went wrong.



The screenshot shows the CSE Trade Summary page. At the top, there's a navigation bar with links like Market, Listed Entities, New Listings, Stockbroker Firms, Investors, News & Events, Rules & Circulars, and About Us. Below this is a section titled 'TRADE SUMMARY' with a blue header. The main content area contains a paragraph about the CSE and its 290 companies. Below this is a table titled 'DAILY SHARE TRADING STATISTICS (SUMMARY)' with columns for Company, Symbol, Share Volume, Trade Volume, Previous Close (Rs.), Open (Rs.), High (Rs.), Low (Rs.), Last Trade (Rs.), Change (Rs.), and Change (%). The table shows data for ABANS ELECTRICALS PLC (ABAN.N0000) with a share volume of 939 and a trade volume of 18. The previous close was 1,042.25, and the open was 1,050.00. The high was 1,061.00, and the low was 1,026.00. The last trade was 1,058.25, with a change of +16.00 (1.54%).

Company	Symbol	Share Volume	Trade Volume	Previous Close (Rs.)	Open (Rs.)	High (Rs.)	Low (Rs.)	Last Trade (Rs.)	Change (Rs.)	Change (%)
ABANS ELECTRICALS PLC	ABAN.N0000	939	18	1,042.25	1,050.00	1,061.00	1,026.00	1,058.25	+16.00	1.54

The CSE trade summary page as our Selenium browser saw it - the table is there, because the JavaScript ran.

Why wait for the table instead of just sleeping?

You could write `time.sleep(10)` and hope the table has appeared by then. But on a fast connection that wastes most of the 10 seconds, and on a slow one 10 seconds might not be enough. `WebDriverWait` carries on the moment the table appears - never sooner, never later than it needs to.

Think of it like this. waiting for a bus by watching the road, instead of setting a timer for 10 minutes. You get on as soon as it arrives, and you never leave before it comes.

Part 3 - read_table()

```
def read_table(table):
    headings = [cell.get_text(strip=True)
                for cell in table.find_all("th")]

    rows = []
    for row in table.find_all("tr"):
        cells = row.find_all("td")
        if not cells:
            continue # this is the heading row
        rows.append([cell.get_text(strip=True) for cell in cells])

    if not rows:
        return pd.DataFrame()

    if len(headings) != len(rows[0]):
        headings = [f"Column {i + 1}" for i in range(len(rows[0]))]

    return pd.DataFrame(rows, columns=headings)
```

This turns an HTML table into a pandas table, using the Week 3 idea of how tables are built: <th> cells are headings, <tr> is a row, and <td> cells are the data.

- The headings are collected from every <th> cell.
- Each row's <td> cells become one list. The heading row has no <td> cells, so continue skips it.
- If the number of headings does not match the number of cells, the columns are simply numbered, so no rows are lost.

What you see when you run it

Try it yourself. run your file on its own (it takes about 5 seconds):

```
python -m src.selenium_scraper
```

```
Browser      : Edge
HTML size    : 142,407 bytes
Words found  : 4,121 characters
Tables       : 1
Time taken   : 5.3 seconds
```

	requests (Nuhan's file)	Selenium (your file)
HTML	17,898 bytes	142,407 bytes
Words found	24 characters	4,121 characters
Tables	0	1

Same address, same BeautifulSoup code afterwards. The only difference is that the browser ran the JavaScript first.

Be careful: Selenium's browser makes its own requests, so they do **not** go through Nuhan's `fetch()` - `robots.txt` is not checked by our code for them, and they are not in the request log. We give the browser our honest User-Agent, load only this one page, and the page is allowed by `robots.txt`. If asked, say this plainly: it is the one exception to 'every request goes through `fetch()`'.

Remember: Selenium works because it runs a real browser, which runs the JavaScript. It is not our main method because it is slower, much heavier for CSE's server, and only gets the first page of about 25 companies - the API gives all of them in one request.

Your files: config.py and warm_cache.py

config.py - every setting in one place

config.py holds every setting the project uses. Nothing in it does anything by itself - it just stores values that the other files read. If CSE ever changes an address, or we want a longer delay, we change one line here instead of hunting through ten files.

Setting	Value	Used by
BASE_URL	https://www.cse.lk	Everything
API_BASE	https://www.cse.lk/api	api_client.py
CDN_BASE	https://cdn.cse.lk/cmt/	circulars() - the PDF server
ENDPOINTS	The 11 data addresses, and GET or POST	call()
USER_AGENT	Our honest name	ethics.py, Selenium
DELAY	1.5	ethics.py - the pause
TIMEOUT	30	Give up waiting for a reply after 30 s
OCR_TEXT_THRESHOLD	50	pdf_extractor.py
OCR_DPI	200	ocr_extractor.py
NAMES	Index numbers and names	The app and the documents

The folders

```
ROOT_DIR = Path(__file__).parent
DATA_DIR = ROOT_DIR / "data"
PDF_DIR = DATA_DIR / "pdfs"
CACHE_DIR = DATA_DIR / "cache"
...

for _folder in (RAW_DIR, PDF_DIR, IMAGE_DIR, EXPORT_DIR, CACHE_DIR,
                LOG_DIR, DOCS_DIR):
    _folder.mkdir(parents=True, exist_ok=True)
```

- Path(__file__).parent - 'the folder this file is in'. So the project works wherever it is copied to, not only on one computer.
- / joins folder names into a path: DATA_DIR / "pdfs" is data\pdfs.
- The loop creates every folder when the project starts. exist_ok=True means 'if it is already there, that is fine'. So no file ever crashes because a folder is missing.

Names beside index numbers

```
NAMES = {
    "24ada076": "Nuhan",
    "25ada072": "Pasindu",
    "25ada073": "Sasini",
    "25ada141": "Salaama",
}
```

```
def member_label(index):
    name = NAMES.get(index)
    return f"{index} - {name}" if name else index
```

`member_label("25ada072")` gives `25ada072 - Pasindu`. The app uses it for the title bar, the Home table, and the label on each tab. `x if condition else y` is a one-line if: 'the name label if we have a name, otherwise just the index'.

warm_cache.py - collecting a copy before the viva

Every reply `fetch()` receives is saved as a copy. `warm_cache.py` asks for everything once, so every copy exists before a demonstration. Then if the internet fails in the viva room, the app shows the saved copies instead of errors.

```
jobs = [
    ("Market status", api.market_status),
    ("Index", api.aspi),
    ("Share prices", api.trade_summary),
    ...
]

for name, job in jobs:
    try:
        result = job()
        size = len(result) if hasattr(result, "__len__") else 1
        print(f" saved {name:18} ({size} record(s))")
    except Exception as e:
        print(f" failed {name:18} {e}")
```

- Each job is a pair: a name to print, and a method to run later.
- `hasattr(result, "__len__")` - 'can this be counted?' A table or list can; a single word like 'Market Closed' cannot, so it counts as 1.
- `{name:18}` pads the name to 18 characters, so the printed list lines up neatly.
- If one job fails, `except` prints it and the loop carries on with the rest.

Try it yourself. run it before the viva - it takes about a minute:

```
python warm_cache.py
```

Remember: `config.py` stores every setting in one place, and `warm_cache.py` collects a copy of everything so the demonstration works without internet - and the demonstration itself then sends almost no requests to CSE.

How your part fits into the whole project

Every button in the app follows the same path. The lines marked <-- YOU are yours.

1. You click a button in the app	app.py	
2. The app asks for some data	src/api_client.py	<-- YOU
3. Every request goes through fetch()	src/ethics.py	
a. check robots.txt	src/ethics.py	
b. wait 1.5 seconds	src/ethics.py	
c. send it, saying who we are	src/ethics.py	
d. write it in the request log	src/ethics.py	
4. The reply (JSON) becomes a table	src/api_client.py	<-- YOU
5. Documents are found and downloaded	src/crawler.py	
6. Their text is read	src/pdf_extractor.py	
7. Scanned pages are read by OCR	src/ocr_extractor.py	
8. The table is tidied and checked	src/cleaner.py	
9. Charts are drawn	src/analysis.py	
10. Files are saved (CSV, Excel, JSON)	src/storage.py	

Two more pieces sit beside this path rather than on it: `src/static_scraper.py` shows the ordinary requests-and-BeautifulSoup method, and `src/selenium_scraper.py` shows the real-browser method. The Scrapy spider in `scrapy_project/` does the document search a second way, with the Scrapy framework.

Your part is step 2 and step 4. When the app wants data, it calls your `CSEApi`. Your `call()` passes the request to Nuhan's `fetch()` at step 3, and when the JSON comes back, your `make_dataframe()` turns it into the table at step 4. Every other part of the project - Sasini's crawler, the cleaning, the charts, the saved files - starts from the tables your code produces.

Ethics - what every member must know

The assignment says marks are given for **following ethical scraping practices** and for **being able to explain your work**. Ethics was Nuhan's file, but any of us can be asked about it, so here is the short version.

Ethical and unethical scraping

Ethical (what we do)	Unethical (what we avoid)
Only collect what anyone can see without logging in	Getting past logins, paywalls or CAPTCHAs
Follow the website's robots.txt	Ignoring robots.txt
Wait between requests	Firing requests as fast as possible
Collect no personal information	Collecting data about people
Credit the source	Using the data without saying where it came from

robots.txt

robots.txt is a small file a website publishes to tell programs like ours which parts they may visit. CSE's says:

```
User-agent: *  
Disallow:  
Disallow: /cgi-bin/  
Sitemap: https://www.cse.lk/sitemap.xml
```

User-agent: * means the rules are for every program. The next two lines ask programs to keep out of one folder, /cgi-bin/. None of the addresses we use are in that folder. There is no line asking us to wait between requests.

Think of it like this. robots.txt is a sign on a shop door that says 'Staff only beyond this point'. Nothing physically stops you walking past it - you stay out because you respect it.

What our code does about it

1. Every request goes through one function, `fetch()` in `src/ethics.py`. It checks robots.txt first.
2. It waits **1.5 seconds** between requests, even though CSE does not ask us to.
3. It sends a **User-Agent** with our university, course and email.
4. It writes every request to `logs/request_log.csv`.
5. It saves a copy of each reply so we never ask twice.
6. Every exported file names the **Colombo Stock Exchange** as the source.

Remember: 'We only collect public data, we follow robots.txt, we wait 1.5 seconds between requests, we say who we are, and we credit CSE in every file we save.' If you are asked about ethics, start with that sentence.

Showing your part in the viva

The viva is 8 minutes for the whole team: about 3 minutes of presentation, then 5 minutes of questions about what each member did. This chapter gets you ready for your share of it.

Before the viva

1. Double-click `Run_CSE_App.bat` in the project folder. The app opens in your browser. Keep the black window open - closing it stops the app.
2. Check your tab works, as below.
3. Run your files once from the terminal, so you have seen the output yourself.

Showing it in the app

1. Open the **Market Data** tab. Your name is on the label at the top.
2. Press **Get the market data**. Point to the numbers across the top - the index, the number of companies, how many rose and fell - then scroll through the **Share prices** table.
3. Open **The addresses we read the data from** below it. This is the list of CSE data addresses you use.
4. Open the **Companies** tab. Press **Load the company list**, choose a company and press **Get details**.
5. If there is time, show the Network tab in a real browser: open the CSE trade summary page, press F12, and point to the `tradeSummary` request.

Showing it from the terminal

```
python -m src.api_client          # the data addresses
python -m src.selenium_scraper    # a real browser (about 5 s)
python warm_cache.py              # save a copy of everything
```

Your 30-second explanation

Practise saying this out loud until it feels natural. Change the words so they sound like you.

"My part is getting the data. The CSE page is built by JavaScript, so I opened it with F12 and watched the Network tab, and saw the page fetching its prices from addresses like `api slash tradeSummary`. Those return JSON, so my code asks those addresses directly - through Nuhan's `fetch` function, so the rules still apply - and turns each reply into a pandas table. One request gives us all 284 companies. I also wrote the Selenium version, which opens a real browser so the JavaScript runs. It works, but it is slower and only gets the first page of about 25 companies, so the API is our main method."

If the examiner points at your code

These are the parts of your files most likely to be pointed at, and what to say about each:

If they point at...	Say...
<code>self.scrapers.fetch(...)</code> in <code>call()</code>	My requests go through Nuhan's <code>fetch</code> , so <code>robots.txt</code> , the delay and the log apply to them too.

If they point at...	Say...
<code>config.ENDPOINTS</code>	The list of CSE data addresses, and whether each needs GET or POST. Most need POST - a GET gives an error.
<code>get_records()</code>	Different addresses wrap their data differently; this always hands back a plain list.
<code>"reqTradeSummery"</code>	CSE's own spelling. We must use their names exactly.
<code>data={"symbol": symbol}</code>	This address needs to know the company, so we send the symbol as form data with the POST.
<code>/ 1000 in to_datetime()</code>	CSE sends milliseconds since 1970; dividing by 1000 gives seconds, which Python turns into a date.
<code>WebDriverWait(...)</code>	Waits until the table appears instead of guessing a sleep time.
<code>driver.page_source</code>	The page after JavaScript ran - the reason Selenium finds the prices.

Practice questions

These are the same 50 questions as your section of the team guide: about 30 on your own part, then 20 on the project as a whole. Cover each answer with your hand and try to say it first. By now, every answer should make sense - if one does not, go back to the chapter it comes from.

EASY - THE BASICS

1. Which files did you write?

`src/api_client.py`, which gets the data from CSE; `src/selenium_scraper.py`, which opens the site in a real browser; `config.py`, the settings; and `warm_cache.py`, which saves a copy of the data so the app works offline.

2. What is an API, in plain words?

A way for one program to ask another program for data. You send a request, and you get a reply back, usually as JSON. The lecture compared it to a waiter: you tell the waiter what you want, the waiter goes to the kitchen, and the food comes back.

3. Why do websites have an API?

So their own pages can load data without reloading the whole page, so other developers can build things with their data, and so that automated access does not have to go through the website itself, which is heavier.

4. What is JSON?

A text format for structured data, made of names and values, and lists of those. It is easy for programs to read, which is why almost every API replies in it.

5. Which address gives you the main dataset?

`https://www.cse.lk/api/tradeSummary`. One request to it returns today's trading for every listed company - around 290 of them.

6. What is Selenium?

A tool that controls a real web browser from code. It can open pages, click things, type into boxes and read what is on screen, exactly as a person would.

7. Which browser does your code use?

Microsoft Edge, because that is what is installed on this machine. The code tries Edge first, then Chrome, then Firefox, so it works on anybody's computer.

8. What does headless mean?

The browser runs without showing a window. It is faster and it does not interrupt a demonstration by popping up on screen. Our code can do either.

9. What does `warm_cache.py` do?

It collects everything once and saves a copy of every reply to `data/cache/`. Running it before a demonstration means the app still works if the internet is unavailable, and it also means the demonstration itself sends almost no requests to CSE.

10. What is web scraping?

Collecting data from websites automatically with a program, instead of copying it by hand. You send a request for a page, get the content back, find the parts you want, and save them in a structured form such as a CSV file.

11. What website did your group work on, and what did you collect?

The Colombo Stock Exchange, www.cse.lk. We collect share prices for around 290 listed companies, details for individual companies such as market value and yearly high and low, and company announcements together with the PDF documents attached to them.

12. Is the data you collected public?

Yes. Everything we collect can be seen by anybody visiting the site without logging in. CSE publishes it so the investing public can read it. We collect no personal information about any individual - only company figures.

13. What is robots.txt?

A text file websites publish at the top level of the site telling automated programs which pages they may and may not visit. It is a voluntary guideline, not a lock - a program can ignore it. Ethical scrapers choose to follow it.

14. What does CSE's robots.txt say?

It blocks one section, `/cgi-bin/`, and allows everything else. It does not set a `Crawl-delay`, so it does not ask us to wait between requests at all.

15. If the site does not ask you to wait, why do you wait?

Because being allowed to go fast is not a reason to. Sending requests as fast as the computer can manage would put load on their server and gain us nothing. We wait 1.5 seconds between requests.

16. What does the HTTP status code 200 mean?

The request worked. 403 means we are not allowed, 404 means the page does not exist, and 500 means the server had a problem.

17. What is a DataFrame?

A table in pandas, with rows and named columns, a bit like a sheet in Excel. Almost everything we collect ends up in one, because it makes the data easy to sort, filter and save.

MEDIUM - HOW IT WORKS

18. How did you find these addresses?

We opened the CSE site in a browser, pressed F12 to open the developer tools, opened the Network tab, and reloaded the page. That tab lists every request the page makes while it loads, and among them were the calls the page was making to its own backend.

19. Why do those addresses need POST and not GET?

Because that is what the server accepts. Sending a GET returns an error. It is a little unusual, because the requests only read data and GET would be the normal choice, but it is the server's decision and we found it by trying both.

`config.ENDPOINTS` records which method each address needs, so `call()` picks the right one automatically.

```
ENDPOINTS = {
    "tradeSummary": {"method": "POST", ...},
    "allSecurityCode": {"method": "GET", ...},
}
```

20. Walk me through `call()`.

It takes the short name of an address, looks up whether it needs GET or POST in `config.ENDPOINTS`, builds the full address, and hands it to `Scraper.fetch()`.

It also passes `save_as=endpoint`, which is what gives the saved copy a readable filename such as `data/cache/tradeSummary.json`.

```
def call(self, endpoint, data=None):
    method = config.ENDPOINTS.get(endpoint, {}).get("method", "POST")
    url = f"{config.API_BASE}/{endpoint}"
    return self.scrapers.fetch(url, method=method, data=data,
                              save_as=endpoint)
```

21. Why do you need `get_records()`?

Because the replies do not all have the same shape. Some addresses return a plain list of records. Others wrap the list inside a dictionary under a name such as `reqTradeSummary`. A couple return a list inside another list.

`get_records()` handles all of those and always hands back a plain list, so the rest of the code does not have to care which address it came from.

22. How does a JSON reply become a table?

The reply is a list of dictionaries, where each dictionary is one company. `pd.DataFrame()` accepts exactly that - it makes one row per dictionary and one column per key. So it is a single line once `get_records()` has found the list.

23. How does `company_info()` differ from the other methods?

It sends a value with the request. The address needs to know which company you mean, so we pass `data={"symbol": "SAMP.N0000"}` in the POST. The others take no parameters.

The reply also has several sections, and the company details are inside the one called `reqSymbolInfo`, so we take that part out.

24. What does a symbol like `SAMP.N0000` mean?

`SAMP` is the company - Sampath Bank. `N` means voting shares, where `X` would mean non-voting. `0000` is the share class number. A company can appear more than once if it has different classes of share listed.

25. Why is `several_companies()` slow?

Because each company is a separate request, and every request waits its 1.5 seconds first. Fifteen companies takes a little over 20 seconds.

That is deliberate. It is the only loop in the project that sends many requests in a row, so it is where politeness matters most.

26. What is `to_datetime()` for?

CSE sends dates as the number of milliseconds since 1970 - a number like `1786440420412`. That is unreadable, and worse, pandas would treat it as an ordinary number. `to_datetime()` divides by 1000 and converts it into a real date and time.

27. How does Selenium solve the JavaScript problem?

Because it is a real browser, the JavaScript actually runs. The browser loads the page, the scripts fetch the data, and the table gets built.

We then ask the browser for the finished HTML with `driver.page_source`, which is the page after JavaScript. That is the key difference from `response.text` in the requests version.

```
driver.get(url)                # browser loads and runs the page
html = driver.page_source      # the page AFTER JavaScript
soup = BeautifulSoup(html, "html.parser") # then as normal
```

28. What is WebDriverWait, and why not just use sleep?

When the browser first opens the page the table does not exist yet, because JavaScript is still fetching the data. If we read the HTML immediately we would get the same empty page requests gets.

WebDriverWait pauses until the element actually appears, then carries on straight away. A fixed sleep is worse in both directions - too short and it fails on a slow connection, too long and it wastes time on every run.

```
WebDriverWait(driver, 15).until(
    expected_conditions.presence_of_element_located(
        (By.TAG_NAME, "table")))

```

29. Why is driver.quit() inside a finally block?

So it always runs, even if something goes wrong in the middle. If the browser is not closed it keeps running in the background using memory, and after a few failed runs there would be several invisible browsers open.

30. How do you turn the rendered table into a DataFrame?

We find the `<table>` with BeautifulSoup, take the heading cells from the `<th>` tags, and then loop over the `<tr>` rows taking the `<td>` cells with `get_text(strip=True)`. That gives a list of rows, which `pd.DataFrame()` turns into a table.

If the number of headings does not match the number of cells we number the columns instead, so we never lose the rows.

31. Selenium needs a driver. How is that handled?

Since Selenium 4.6 it handles it itself. Selenium Manager works out which browser is installed, downloads the matching driver the first time it runs, and remembers it. Older tutorials tell you to download chromedriver by hand and match the version, which is no longer necessary.

32. Why did requests and BeautifulSoup not work on the CSE site?

Because the page is dynamic. requests downloads the HTML file and stops - it does not run JavaScript. The CSE server sends an almost empty page, and a program inside the browser fetches the prices and puts them on the page afterwards. So the file we download has about 24 characters of text in it and no tables.

33. What is the difference between static and dynamic content?

On a static page the content is already inside the HTML file the server sends, so downloading the file is enough. On a dynamic page the file arrives nearly empty and JavaScript fills it in afterwards, so you need either a real browser or the address the page gets its data from.

34. How did you find the addresses the site uses for its data?

We opened the CSE site in a browser, pressed F12 to open the developer tools, went to the Network tab, and reloaded the page. That tab lists every request the page makes, and it showed the page fetching its own data from addresses like `/api/tradeSummary`.

35. What is JSON, and why is it convenient here?

JSON is a structured text format for data, made of names and values. It is convenient because it is already organised - pandas can turn it straight into a table, with no need to hunt through HTML tags for the numbers.

36. How does your project make sure the ethical rules are followed?

Every request in the whole project goes through one function, `Scraper.fetch()` in `src/ethics.py`. That function checks `robots.txt`, waits the delay, sends the request with our User-Agent, and writes a line to the log. Because there is only one way out to the internet, the rules cannot be skipped by accident in some other file.

37. What does your User-Agent say, and why does that matter?

It gives the university, the course, the group and an email address, instead of pretending to be an ordinary browser. If our requests ever caused a problem, the site administrator could see who was responsible and get in touch.

38. How much load did you put on CSE's servers?

Around 30 to 40 requests for a full run of the app, spaced at least 1.5 seconds apart. Every one is recorded in `logs/request_log.csv`. Somebody browsing the same pages by hand would actually generate more traffic, because each page load also pulls scripts, fonts and images that we never request.

39. Why are there two kinds of PDF in the project?

A text based PDF was made by a computer, so the letters inside it are real text and we can read them straight out. A scanned PDF is a photograph of a printed page - the words are part of a picture, so there is no text inside to extract and it has to go through OCR instead.

HARDER - EXPLAIN AND DEFEND IT

40. Why is using the API better than scraping the page, and is it fair to CSE?

It is better for us because it is faster, more reliable, and gives every company in one request instead of one page at a time.

It is also easier on CSE, which is the part that matters ethically. Loading the page makes their server send the HTML, the JavaScript bundles, the fonts and the images, and then the browser calls that same data address at the end of it anyway. Going straight to the data address skips all of that work.

The Week 6 lecture said it directly: use APIs where available, because they are more reliable and more structured than scraping.

41. What are the limitations of relying on an API?

Rate limits, if the provider sets them. Needing a key or an account for many APIs, although not this one. Only getting the data the provider chose to expose, which may be less than the website shows. And the biggest one for us: an API can change without warning, and ours is not officially documented, so CSE has no reason to tell anybody before changing it.

The way we guard against that is keeping every address in `config.py`, so a change means editing one table.

42. The reply for one company has several sections. How did you know which one to use?

By printing the reply and looking at it. The top level keys were `reqSymbolInfo`, `reqSymbolBetaInfo`, `reqLogo` and `reqTagsLogo`.

The prices and the market value were inside `reqSymbolInfo`, so that is the part we take. The others hold the logo image paths and a volatility figure we do not use.

43. Selenium works. Why is it not your main method?

Three reasons.

It is slow - several seconds against about one for the data address. It is heavy on CSE, because it makes them serve the whole page and all its assets. And it only gets part of the data, because the table on screen is paginated, so one load gives about 25 companies rather than all 290.

We keep it because it proves the browser approach works, and because it would be our fallback if the data addresses ever changed.

44. How would you use Selenium to get all 290 companies?

You would have to work through the pagination - find the next page button, click it, wait for the table to reload, read it, and repeat until the button is disabled, collecting the rows as you go.

That is perfectly doable, but it means one page load per 25 companies, so roughly twelve full page loads with a wait after each. Against a single request that returns everything, it is hard to justify.

45. Why does your Selenium code set the same User-Agent as the rest of the project?

So CSE sees one consistent visitor rather than two different unknown programs. If we identify ourselves honestly in the API requests but let the browser send its default Edge User-Agent, the same project would look like two separate things in their logs, and only one of them would be identifiable.

46. What is the difference between what `requests` returns and what Selenium returns for the same page?

The same address gives about 25,000 bytes with roughly 24 characters of text and no tables through `requests`, and about 140,000 bytes with roughly 4,000 characters of text and a real table through Selenium.

Same URL, same parsing code afterwards. The only difference is that one of them ran the JavaScript.

47. Is web scraping legal?

It is not automatically legal or illegal - it depends on the data, the website's terms, and what you do with the result.

We stayed well on the safe side: only public company information, no personal data, robots.txt followed, no security measure bypassed, the source credited in every file, and academic use only. We also did not ask CSE for written permission, which would be the right next step if we wanted to publish or sell anything based on this.

48. Is it acceptable to use an address that is not officially documented?

We think so, for three reasons.

It needs no login and no key, so it is exactly the same request the page makes for any visitor. `robots.txt` does not ask us to stay away from it. And it is actually less work for their server than loading the whole page, because loading the page makes them send the HTML, the JavaScript, the fonts and the images, and the browser then requests that same data address at the end of it anyway.

The honest limitation is that an undocumented address can change without warning, and that is CSE's right.

49. Did you get past any CAPTCHAs or logins?

No, and we would not. Everything we collect is visible without logging in, and we never met a CAPTCHA because we never behaved in a way that would set one off.

A CAPTCHA is a website asking whether you are a person. Getting around one means answering yes when the honest answer is no. Being able to defeat a control is not the same as being allowed to.

50. What would break this project, and how bad would it be?

CSE changing the addresses their site uses to load data. They are not officially documented, so they could change at any time without notice.

It would not be a disaster. Every address is listed in one place, in `config.py`, so fixing it means editing that table rather than hunting through the whole project. We also save a copy of every reply, so a demonstration still works from the saved data even if the live site has changed.

Word list

Every special word used in this document, from A to Z.

Word	What it means
API	Application Programming Interface - an address a program can ask for data.
CSV	A simple table file: one row per line, commas between columns.
Database	Where a website keeps all its information.
DataFrame	A table in pandas, with rows and named columns.
DOM	The page as it is after JavaScript has run.
Driver	A small program that passes Selenium's orders to a browser.
Dynamic page	A page whose content is added by JavaScript later.
Endpoint	One particular API address, such as tradeSummary.
Epoch time	A moment stored as the number of milliseconds since 1 January 1970.
Explicit wait	Waiting until something appears on a page, instead of for a fixed time.
Form data	Values sent along with a POST request.
Function	A named set of steps that can be called to do a job.
GET	A request that just asks for something.
Headless	Running a browser with no window.
HTML	The language web pages are written in, using tags.
JavaScript	A language that runs inside the browser and can change the page after it loads.
JSON	A tidy text format for data: names and values.
Library	Code somebody else wrote that we can import and use.
Market cap	A company's total value - share price times number of shares.
Network tab	The part of the browser's developer tools that lists every request a page makes.
PDF	A document file that looks the same on every computer.
POST	A request that sends some information along with it.
Request	A message asking a server for something.
Response	The server's reply to a request.
robots.txt	A file where a website says which parts programs may visit.
Selenium	A tool that lets a program control a real web browser.
Server	A computer that stores a website and answers requests.
Share	A small piece of a company that people can buy and sell.
Static page	A page whose content is already in the HTML file.

Word	What it means
Status code	A number on each reply saying how it went - 200 means OK.
Symbol	A company's short trading code, such as SAMP.N0000.
Tag	A label in HTML, such as <title> or <table>.
Turnover	The total money that changed hands in a day.
URL	The address of a page or file on the web.
User-Agent	The part of a request that says which program sent it.
Variable	A name that holds a value.